CSS

From Beginner to Professional

A Complete Student Textbook

Style the Web with Confidence

Welcome to CSS

CSS (Cascading Style Sheets) is the language that makes the web beautiful. Every color, font, layout, and animation on a website is CSS. This textbook covers every major CSS concept with real-world code you can run immediately in any browser.

How to Use This Book

- Create an HTML file and a linked CSS file — all examples run directly in the browser.
- Open Chrome DevTools (F12 → Elements tab) to inspect and live-edit styles.
- Experiment: every code block is a starting point, not a finished product.

Your Starter Template

Save the code below as index.html and styles.css — use this for every exercise:

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS Practice</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <h1>Hello, CSS!</h1>
  <p class="intro">Style me!</p>
</body>
</html>

/* styles.css */
body {
  font-family: Arial, sans-serif;
  max-width: 900px;
  margin: 0 auto;
  padding: 20px;
}
h1 { color: #E44D26; }
.intro { color: #264DE4; font-size: 1.2rem; }
```

💡 **TIP**

Open any webpage, right-click an element → 'Inspect', then edit its CSS live in the Styles panel. Best learning tool ever.

CHAPTER 1

Getting Started with CSS

Adding Styles, Selectors & the Cascade

Three Ways to Add CSS

CSS can be added inline, via a <style> tag, or (best practice) a linked external file.

```
<!-- 1. External stylesheet — best practice (reusable across all pages) -->
<link rel="stylesheet" href="styles.css">

<!-- 2. Internal <style> tag — use for single-page demos -->
<style>
  h1 { color: tomato; }
</style>

<!-- 3. Inline style — last resort, hardest to maintain -->
<h1 style="color: tomato; font-size: 2rem;">Hello</h1>
```

🌐 **REAL WORLD**

Real-World: Big sites like airbnb.com use thousands of external CSS files bundled together. Inline styles are avoided because they can't be reused and override everything.

CSS Rule Anatomy

A CSS rule has a selector (who to style) and a declaration block (how to style it).

```
/* selector  { property: value; } */

h1 {                          /* selector */
  color: #264DE4;             /* declaration */
  font-size: 2rem;            /* declaration */
  text-align: center;         /* declaration */
}

/* Multiple selectors — apply same style to both */
h1, h2, h3 {
  font-family: 'Arial', sans-serif;
  font-weight: bold;
}
```

Basic Selectors

Selector	Example	Selects
`*`	* { margin: 0; }	Every element on the page
`element`	p { ... }	All <p> elements

`.class`	.card { ... }	All elements with class="card"
`#id`	#header { ... }	Element with id="header"
`el.class`	p.intro { ... }	Only <p> elements with class intro
`el1, el2`	h1, h2 { ... }	Both h1 AND h2 elements

Attribute Selectors

Target elements based on their HTML attributes — powerful for styling form inputs and links.

```
/* Exact match */
input[type="text"]  { border: 2px solid blue; }
input[type="email"] { border: 2px solid green; }

/* Starts with — target all https links */
a[href^="https"] { color: green; }

/* Ends with — style PDF links */
a[href$=".pdf"]  { color: red; }
a[href$=".pdf"]::after { content: " (PDF)"; }

/* Contains — select inputs with 'email' anywhere in name */
input[name*="email"] { background: #fef3e2; }

/* Has the attribute at all */
img[alt] { border: 2px solid green; }   /* Images WITH alt text */
img:not([alt]) { border: 2px solid red; } /* Images MISSING alt */
```

Combinators — Relationship Selectors

```
/* Descendant (space) — any level deep */
.card p { color: #555; }          /* all <p> inside .card */

/* Child (>) — direct children only */
ul > li { list-style: square; }   /* only direct <li> of <ul> */

/* Adjacent sibling (+) — immediately after */
h2 + p { font-size: 1.1em; }      /* first <p> after any <h2> */

/* General sibling (~) — all siblings after */
h2 ~ p { color: #444; }           /* all <p> after h2 at same level */

/* Real-world: Style form elements after a label */
label + input {
  display: block;
  margin-top: 4px;
  padding: 8px 12px;
  border: 1px solid #ccc;
  border-radius: 4px;
}
```

Pseudo-Classes — Style Based on State

```
/* Link states — must be in this order: LVHA */
a:link    { color: blue; }         /* unvisited */
a:visited { color: purple; }       /* visited */
a:hover   { color: red; }          /* mouse over */
a:active  { color: orange; }       /* being clicked */

/* Form states */
input:focus   { outline: 2px solid #264DE4; box-shadow: 0 0 0 4px
rgba(38,77,228,0.2); }
input:valid   { border-color: green; }
input:invalid { border-color: red; }
input:checked + label { font-weight: bold; }

/* Structural pseudo-classes */
li:first-child  { font-weight: bold; }   /* first item */
li:last-child   { border-bottom: none; } /* last item */
li:nth-child(2) { background: #f3f4f6; } /* exact position */
li:nth-child(odd)  { background: #f9f9f9; } /* zebra stripe! */
li:nth-child(even) { background: #ffffff; }

/* Negate — style all EXCEPT */
input:not([type="submit"]) { border: 1px solid #ccc; }
```

🌐 **REAL WORLD**

Real-World: Every e-commerce product list uses :nth-child(odd/even) for zebra-striped tables. Every form uses :focus and :valid/:invalid for user feedback.

CHAPTER 2

The Box Model

Margin, Border, Padding & box-sizing

The Box Model

Every HTML element is a rectangular box with four layers: content → padding → border → margin. Understanding this is the #1 CSS skill.

Box Model Layers (outside → in)

MARGIN — space OUTSIDE the border (transparent, separates from neighbors)
BORDER — the visible edge around the element
PADDING — space INSIDE the border (inherits background color)
CONTENT — your text, images, or child elements

```
.product-card {
  /* Content area */
  width: 300px;
  height: 200px;

  /* Padding: space inside (shows background) */
  padding: 20px;

  /* Border: visible edge */
  border: 2px solid #e44d26;

  /* Margin: push away neighbors */
  margin: 16px;

  /* Total width (default): 300 + 40 + 4 + 32 = 376px ← CONFUSING! */
}
```

box-sizing: border-box — The Essential Fix

By default, padding and border ADD to the declared width. border-box makes width INCLUDE padding and border. Always use it.

```
/* ❌ Default (content-box) */
.box {
  width: 300px;
  padding: 20px;
  border: 2px solid black;
  /* Actual width = 344px! */
}
```

```
/* ✅ border-box (use this always) */
*, *::before, *::after {
  box-sizing: border-box;
}
.box {
  width: 300px;
  padding: 20px;
  border: 2px solid black;
  /* Actual width = 300px */
}
```

Margin

Margin creates space OUTSIDE an element. Use shorthand for concise code.

```
/* Shorthand — top right bottom left (clockwise from top) */
.el { margin: 10px 20px 30px 40px; }  /* T R B L */
.el { margin: 10px 20px; }             /* TB=10, LR=20 */
.el { margin: 10px; }                  /* all sides = 10px */

/* Individual sides */
.el { margin-top: 20px; margin-bottom: 20px; }

/* Auto margin — center a block horizontally */
.container {
  max-width: 1200px;
  margin: 0 auto;  /* 0 top/bottom, auto left/right = centered */
}

/* Negative margin — pull element closer or overlap */
.pulled-up { margin-top: -20px; }

/* ⚠️ Margin Collapse! Adjacent vertical margins merge — only keep larger value */
h1 { margin-bottom: 30px; }
p  { margin-top: 20px; }
/* Gap between h1 and p is 30px (not 50px) */
```

Padding

```
/* Shorthand — same syntax as margin */
.btn {
  padding: 12px 24px;  /* 12px top/bottom, 24px left/right */
}

/* Typical button padding pattern */
.btn-sm  { padding: 6px  14px; }
.btn-md  { padding: 10px 20px; }
.btn-lg  { padding: 14px 28px; }

/* Note: padding % is relative to parent WIDTH, even for top/bottom */
.aspect-ratio-box {
  width: 100%;
  padding-top: 56.25%;  /* Creates 16:9 aspect ratio! */
  position: relative;
}
```

Borders

```
/* Full shorthand: width style color */
.card { border: 1px solid #e5e7eb; }

/* Border styles */
.el { border-style: solid; }    /* ——— */
.el { border-style: dashed; }   /* - - */
.el { border-style: dotted; }   /* ... */
```

```
.el { border-style: double; }   /* ═══ */
.el { border-style: none; }     /* no border */

/* Individual sides */
.card-bottom { border-bottom: 3px solid #e44d26; }

/* Border radius — rounded corners */
.pill   { border-radius: 9999px; }  /* pill shape */
.circle { border-radius: 50%; }     /* perfect circle */
.card   { border-radius: 8px; }     /* modern card */

/* Asymmetric radius: top-left top-right bottom-right bottom-left */
.custom { border-radius: 20px 4px 20px 4px; }

/* Real-world card */
.product-card {
  border: 1px solid #e5e7eb;
  border-radius: 12px;
  overflow: hidden;  /* clip image to rounded corners */
}
```

📝 Project: Styled Product Card

```
/* HTML:
  <div class='card'>
    <img class='card-img' src='product.jpg' alt='Product'>
    <div class='card-body'>
      <h3 class='card-title'>Wireless Headphones</h3>
      <p class='card-price'>$79.99</p>
      <button class='btn-buy'>Add to Cart</button>
    </div>
  </div>
*/

*, *::before, *::after { box-sizing: border-box; }

.card {
  width: 280px;
  border: 1px solid #e5e7eb;
  border-radius: 12px;
  overflow: hidden;
  box-shadow: 0 4px 6px rgba(0,0,0,0.07);
}
.card-img  { width: 100%; height: 200px; object-fit: cover; }
.card-body { padding: 16px; }
.card-title { margin: 0 0 8px; font-size: 1.1rem; }
.card-price { margin: 0 0 16px; color: #e44d26; font-weight: bold; font-size:
1.2rem; }
.btn-buy {
  display: block;
  width: 100%;
  padding: 10px;
  background: #264DE4;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
```

```
}
.btn-buy:hover { background: #1a3bc4; }
```

CHAPTER 3

Colors, Typography & Backgrounds

The Visual Foundation of Every Page

Colors — Every Format

CSS supports named colors, hex, rgb, rgba, hsl, and hsla. Modern CSS also has oklch for better color science.

```
/* Named colors — 140 available */
.el { color: tomato; background: cornflowerblue; }

/* Hexadecimal — #RRGGBB or #RGB shorthand */
.el { color: #E44D26; }     /* CSS orange */
.el { color: #fff; }        /* white (#ffffff) */
.el { color: #000; }        /* black (#000000) */

/* rgb(red, green, blue) — values 0–255 */
.el { color: rgb(228, 77, 38); }

/* rgba — same + alpha (0=transparent, 1=opaque) */
.overlay { background: rgba(0, 0, 0, 0.5); } /* semi-transparent black */

/* hsl(hue, saturation%, lightness%) */
.el { color: hsl(14, 75%, 52%); }  /* orange */
.el { color: hsl(14, 75%, 30%); }  /* darker orange */
.el { color: hsl(14, 75%, 80%); }  /* lighter orange */

/* CSS Custom Properties for a color system */
:root {
  --color-primary:   #264DE4;
  --color-danger:    #E44D26;
  --color-success:   #047857;
  --color-text:      #111827;
  --color-muted:     #6B7280;
  --color-bg:        #F9FAFB;
}
.btn-primary { background: var(--color-primary); }
.error-msg   { color: var(--color-danger); }
```

Typography — Text Styling

Typography is 90% of web design. Get these properties right and your page looks professional immediately.

```
body {
  /* Font stack: try each, use first available */
  font-family: 'Inter', 'Segoe UI', Arial, sans-serif;
  font-size: 16px;      /* base size */
  line-height: 1.6;     /* 1.5–1.7 is ideal for readability */
  color: #111827;
}
```

```
/* Font size — rem scales with browser settings (accessible) */
h1 { font-size: 2.5rem; }    /* 40px at default */
h2 { font-size: 2rem;   }    /* 32px */
h3 { font-size: 1.5rem; }    /* 24px */
p  { font-size: 1rem;   }    /* 16px */
small { font-size: 0.875rem; } /* 14px */

/* Font weight */
.thin    { font-weight: 300; }
.normal  { font-weight: 400; }
.medium  { font-weight: 500; }
.bold    { font-weight: 700; }
.black   { font-weight: 900; }

/* Text alignment */
.center { text-align: center; }
.right  { text-align: right; }
.justify { text-align: justify; } /* newspapers */

/* Text decoration */
a { text-decoration: none; }              /* remove underline */
.strikethrough { text-decoration: line-through; }
.underline     { text-decoration: underline; }

/* Text transform */
.uppercase { text-transform: uppercase; }
.capitalize { text-transform: capitalize; }

/* Letter & word spacing */
.heading-spaced { letter-spacing: 0.1em; }
.loose          { word-spacing: 0.2em; }

/* Overflow — handle long text */
.truncate {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;  /* shows '...' */
  max-width: 200px;
}

/* Text shadow */
.hero-title {
  text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
}
```

Google Fonts — Free Custom Fonts

```
<!-- Add to <head> in HTML -->
<link href="https://fonts.googleapis.com/css2?
family=Inter:wght@400;600;700&family=Playfair+Display:wght@700&display=swap"
rel="stylesheet">

/* Then use in CSS */
body     { font-family: 'Inter', sans-serif; }
h1, h2   { font-family: 'Playfair Display', serif; }
```

Backgrounds

```
/* Solid color */
.hero { background-color: #1a1a2e; }

/* Background image */
.hero {
  background-image: url('hero.jpg');
  background-size: cover;      /* cover entire element */
  background-position: center; /* center the image */
  background-repeat: no-repeat;
}

/* Shorthand: color image repeat position / size */
.hero {
  background: #1a1a2e url('hero.jpg') no-repeat center / cover;
}

/* Linear gradient */
.gradient {
  background: linear-gradient(135deg, #264DE4 0%, #E44D26 100%);
}
.fade-down {
  background: linear-gradient(to bottom, rgba(0,0,0,0) 0%, rgba(0,0,0,0.7) 100%);
}

/* Radial gradient */
.spotlight {
  background: radial-gradient(circle at center, #264DE4, #0a0a1a);
}

/* Multiple backgrounds — stacked, first is on top */
.multi {
  background:
    url('dots.png') repeat,
    linear-gradient(135deg, #264DE4, #E44D26);
}

/* Background size options */
.cover   { background-size: cover; }    /* fills, may crop */
.contain { background-size: contain; }  /* fits, may have gaps */
.exact   { background-size: 400px 200px; }

/* Fixed background — parallax effect */
.parallax {
  background-attachment: fixed;
  background-size: cover;
}
```

📝 Project: Hero Section

```
/* HTML:
  <section class='hero'>
    <div class='hero-content'>
```

```
      <h1 class='hero-title'>Build Amazing Websites</h1>
      <p class='hero-sub'>Learn CSS and style the web</p>
      <a class='hero-btn' href='#'>Get Started</a>
    </div>
  </section>
*/

.hero {
  background: linear-gradient(135deg, #1a1a2e 0%, #264DE4 100%);
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  text-align: center;
  padding: 40px 20px;
}
.hero-content { max-width: 600px; }
.hero-title {
  font-size: 3.5rem;
  font-weight: 800;
  color: white;
  line-height: 1.1;
  margin-bottom: 20px;
}
.hero-sub {
  font-size: 1.25rem;
  color: rgba(255,255,255,0.8);
  margin-bottom: 32px;
}
.hero-btn {
  display: inline-block;
  padding: 14px 32px;
  background: #E44D26;
  color: white;
  text-decoration: none;
  border-radius: 9999px;
  font-weight: 700;
  font-size: 1rem;
  letter-spacing: 0.05em;
  transition: transform 0.2s, background 0.2s;
}
.hero-btn:hover {
  background: #c43a16;
  transform: translateY(-2px);
}
```

CHAPTER 4

Flexbox

One-Dimensional Layouts Made Simple

Flexbox is the most used CSS layout tool. It handles rows and columns, spacing, and alignment with minimal code. Master this and you can build almost any layout.

The Mental Model

Flexbox Quick Map

flex-direction → which axis? row (→) or column (↓)
justify-content → align along MAIN axis (row=horizontal, col=vertical)
align-items → align along CROSS axis (row=vertical, col=horizontal)
flex-wrap → allow items to wrap to next line?
gap → space BETWEEN items (replaces margin hacks)

```
.container {
  display: flex;              /* activate flexbox */
  flex-direction: row;        /* row | row-reverse | column | column-reverse */
  justify-content: flex-start;/* flex-start | center | flex-end | space-between |
space-around | space-evenly */
  align-items: stretch;       /* stretch | flex-start | center | flex-end | baseline
*/
  flex-wrap: nowrap;          /* nowrap | wrap | wrap-reverse */
  gap: 16px;                  /* gap between all items */
}
```

Common Flexbox Patterns

1. Perfectly Center Anything

```
.center-everything {
  display: flex;
  justify-content: center;   /* horizontal center */
  align-items: center;       /* vertical center */
  min-height: 100vh;         /* full viewport height */
}
/* Works for dialogs, loading screens, hero sections */
```

2. Navigation Bar

```
/* HTML:
  <nav class='navbar'>
    <a class='logo' href='/'>Brand</a>
    <ul class='nav-links'>
      <li><a href='/'>Home</a></li>
      <li><a href='/about'>About</a></li>
```

```
    </ul>
    <button class='nav-cta'>Sign Up</button>
  </nav>
*/
.navbar {
  display: flex;
  justify-content: space-between;  /* logo left, links center/right */
  align-items: center;
  padding: 16px 32px;
  background: white;
  box-shadow: 0 1px 3px rgba(0,0,0,0.1);
}
.nav-links {
  display: flex;
  gap: 24px;
  list-style: none;
  margin: 0; padding: 0;
}
```

3. Card Grid with flex-wrap

```
.card-grid {
  display: flex;
  flex-wrap: wrap;    /* wrap cards to next line */
  gap: 20px;
}
.card {
  flex: 1 1 280px;  /* grow, shrink, min-width 280px */
  /* Cards fill available space, min 280px, wrap if needed */
}
```

4. Sticky Footer

```
body {
  display: flex;
  flex-direction: column;
  min-height: 100vh;         /* full page height */
}
main { flex: 1; }            /* grows to fill space → pushes footer down */
footer { background: #1a1a2e; color: white; padding: 20px; }
```

5. Holy Grail Layout

```
/* Classic layout: header, sidebar + main + sidebar, footer */
.layout {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}
.layout-body {
  display: flex;
  flex: 1;
}
.sidebar { width: 250px; background: #f3f4f6; padding: 20px; }
.main    { flex: 1;      padding: 20px; }
```

```
header, footer { padding: 16px; background: #1a1a2e; color: white; }
```

Flex Item Properties

```
.item {
  /* flex: grow shrink basis */
  flex: 1;         /* shorthand for: 1 1 0% — grow equally */
  flex: 0 0 200px; /* fixed 200px, don't grow or shrink */
  flex: 2;         /* take twice as much space as flex:1 siblings */

  /* Override alignment for THIS item only */
  align-self: center;      /* center this item on cross axis */
  align-self: flex-end;    /* push to bottom */

  /* Order — visually reorder without changing HTML */
  order: -1;    /* first visually */
  order: 1;     /* last visually */
}

/* Practical: push last item to far right */
.navbar .spacer { flex: 1; }
/* OR: */
.last-item { margin-left: auto; }
```

📝 Project: Responsive Product Cards

```
/* Full responsive layout that adapts to screen size */
.shop-grid {
  display: flex;
  flex-wrap: wrap;
  gap: 24px;
  padding: 24px;
  max-width: 1200px;
  margin: 0 auto;
}
.product-card {
  flex: 1 1 250px;            /* min 250px, flex equally */
  max-width: 320px;           /* cap at 320px */
  border: 1px solid #e5e7eb;
  border-radius: 12px;
  overflow: hidden;
  display: flex;
  flex-direction: column;     /* vertical layout inside card */
}
.product-image { width: 100%; aspect-ratio: 4/3; object-fit: cover; }
.product-body {
  display: flex;
  flex-direction: column;
  flex: 1;
  padding: 16px;
}
.product-price { margin-top: auto; }  /* push price to bottom */
```

CHAPTER 5

CSS Grid

Two-Dimensional Page Layouts

Grid is CSS's two-dimensional layout system. Where Flexbox handles one axis (row OR column), Grid handles both simultaneously. Use Grid for page layouts; use Flexbox for component-level layouts.

Grid Basics

```
.container {
  display: grid;

  /* Define columns */
  grid-template-columns: 200px 1fr 1fr;  /* fixed + two equal */
  grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr)); /* responsive! */

  /* Define rows */
  grid-template-rows: auto 1fr auto; /* header, main, footer */

  /* Gutters between cells */
  gap: 20px;               /* row and column gap */
  row-gap: 16px;
  column-gap: 24px;
}

/* fr = fraction of available space */
/* 1fr 2fr 1fr → 25% 50% 25% */
```

Placing Items on the Grid

```
.grid { display: grid; grid-template-columns: repeat(4, 1fr); }

/* Span across columns */
.header  { grid-column: 1 / -1; }          /* span all columns */
.sidebar { grid-column: 1 / 2; }           /* column 1 only */
.main    { grid-column: 2 / -1; }          /* columns 2–end */
.wide    { grid-column: span 2; }          /* span 2 columns */

/* Span rows */
.tall    { grid-row: span 2; }             /* span 2 rows */
.feature { grid-column: 1/3; grid-row: 1/3; } /* 2×2 block */

/* Named grid areas — most readable approach */
.layout {
  display: grid;
  grid-template-areas:
    'header  header  header'
    'sidebar main    main  '
    'footer  footer  footer';
```

```
  grid-template-columns: 250px 1fr 1fr;
  grid-template-rows: auto 1fr auto;
  min-height: 100vh;
}
header  { grid-area: header; }
aside   { grid-area: sidebar; }
main    { grid-area: main; }
footer  { grid-area: footer; }
```

Responsive Grid — The Power Pattern

```
/* Auto-responsive cards — no media queries needed! */
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
  gap: 24px;
}
/*
 * auto-fill: create as many columns as fit
 * minmax(280px, 1fr): each column min 280px, max equal share
 * Result: 1 col on mobile, 2–3 on tablet, 4+ on desktop
 */

/* Image gallery — masonry-like */
.gallery {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 200px;
  gap: 8px;
}
.gallery .wide  { grid-column: span 2; }
.gallery .tall  { grid-row:    span 2; }
```

Grid Alignment

```
.grid {
  /* Align ALL items */
  justify-items: start | center | end | stretch;   /* horizontal in cell */
  align-items:   start | center | end | stretch;   /* vertical in cell */

  /* Align the whole grid track when it doesn't fill container */
  justify-content: start | center | end | space-between | space-around;
  align-content:   start | center | end | space-between;
}

/* Override for single item */
.featured {
  justify-self: center;  /* center this cell horizontally */
  align-self: center;    /* center this cell vertically */
}
```

📝 Project: Magazine Layout

```
/* HTML:
  <div class='magazine'>
    <header class='mag-header'>DESIGN WEEKLY</header>
    <article class='feature'>Main Feature Story</article>
    <aside class='secondary-1'>Story 2</aside>
    <aside class='secondary-2'>Story 3</aside>
    <aside class='sidebar'>Sidebar</aside>
  </div>
*/
.magazine {
  display: grid;
  grid-template-columns: 2fr 1fr 200px;
  grid-template-rows: 80px 1fr 1fr;
  gap: 16px;
  max-width: 1100px;
  margin: 0 auto;
  padding: 16px;
  min-height: 100vh;
}
.mag-header   { grid-column: 1 / -1; background: #1a1a2e; color: white; display:
flex; align-items: center; padding: 0 24px; }
.feature      { grid-column: 1; grid-row: 2 / 4; background: #f3f4f6; padding: 24px;
border-radius: 8px; }
.secondary-1  { grid-column: 2; background: #dbeafe; padding: 16px; border-radius:
8px; }
.secondary-2  { grid-column: 2; background: #d1fae5; padding: 16px; border-radius:
8px; }
.sidebar      { grid-column: 3; grid-row: 2 / 4; background: #f9f9f9; padding: 16px;
border-radius: 8px; }
```

CHAPTER 6

Positioning & the Visual Stack

Static, Relative, Absolute, Fixed & Sticky

Positioning lets you take elements out of the normal document flow and place them anywhere. It's used for dropdowns, modals, tooltips, and sticky headers.

The Five Position Values

Value	Behavior	Common Use
`static`	Default. In normal flow, top/left/etc. have no effect.	Default state
`relative`	In flow, but top/left offset from its OWN position.	Tooltip anchor, nudge
`absolute`	Out of flow, positioned relative to nearest non-static parent.	Dropdowns, badges, tooltips
`fixed`	Out of flow, positioned relative to VIEWPORT. Stays on scroll.	Sticky nav, cookie banner
`sticky`	In flow until threshold, then sticks to viewport.	Sticky table headers

Relative Positioning

```
/* Stays in flow, offset from its normal position */
.nudged {
  position: relative;
  top: 5px;   /* moves DOWN 5px */
  left: 10px; /* moves RIGHT 10px */
}

/* Most important use: makes it a positioning CONTEXT for absolute children */
.card {
  position: relative; /* anchor for the badge below */
}
.badge {
  position: absolute;
  top: -8px;
  right: -8px;
  /* Positioned relative to .card, not the whole page */
}
```

Absolute Positioning

```
/* Removed from flow, positioned relative to nearest positioned ancestor */

/* Common pattern: notification badge on button */
```

```
.btn-wrapper {
  position: relative;  /* <- important! */
  display: inline-block;
}
.notification-badge {
  position: absolute;
  top: -6px;
  right: -6px;
  width: 20px;
  height: 20px;
  background: red;
  color: white;
  border-radius: 50%;
  font-size: 11px;
  display: flex;
  align-items: center;
  justify-content: center;
}

/* Centering absolutely positioned element */
.centered-overlay {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%); /* shift back by half its size */
}
```

Fixed Positioning

```
/* Stays in viewport even when scrolling */

/* Sticky navigation bar */
.navbar {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;           /* or width: 100%; */
  height: 60px;
  background: white;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
  z-index: 100;
}
body { padding-top: 60px; } /* prevent content hiding under navbar */

/* Cookie consent banner */
.cookie-banner {
  position: fixed;
  bottom: 0;
  left: 0;
  right: 0;
  background: #1a1a2e;
  color: white;
  padding: 16px 24px;
  z-index: 999;
}
```

Sticky Positioning

```
/* Sticks to viewport edge after scrolling past threshold */
.sticky-header {
  position: sticky;
  top: 0;           /* sticks at 0px from top */
  background: white;
  z-index: 10;
}

/* Sticky table header */
thead tr th {
  position: sticky;
  top: 0;
  background: #1a1a2e;
  color: white;
  z-index: 1;
}

/* Sidebar that sticks when scrolling long content */
.sticky-sidebar {
  position: sticky;
  top: 80px; /* 80px from top (leave room for navbar) */
  max-height: calc(100vh - 100px);
  overflow-y: auto;
}
```

z-index — The Stacking Order

```
/* z-index controls which element is ON TOP when they overlap */
/* Higher number = closer to user (on top) */
/* z-index only works on positioned elements (not static) */

/* Typical z-index scale */
:root {
  --z-base:    1;
  --z-dropdown: 100;
  --z-sticky:  200;
  --z-overlay: 300;   /* modal backdrop */
  --z-modal:   400;   /* modal content */
  --z-toast:   500;   /* notifications */
}

.modal-backdrop {
  position: fixed;
  inset: 0;           /* shorthand for top/right/bottom/left: 0 */
  background: rgba(0,0,0,0.5);
  z-index: var(--z-overlay);
}
.modal {
  position: fixed;
  top: 50%; left: 50%;
  transform: translate(-50%, -50%);
  z-index: var(--z-modal);
  background: white;
  padding: 32px;
  border-radius: 12px;
```

```
}
```

🌐 **REAL WORLD**

Real-World: Modal dialogs, dropdown menus, tooltips, notification badges, floating action buttons, cookie banners — all use position:fixed or position:absolute with z-index.

CHAPTER 7

Responsive Design

Media Queries & Mobile-First CSS

Responsive design makes your website look great on all screen sizes — from phones to 4K monitors. The key tools are media queries, relative units, and flexible layouts.

Media Queries

Media queries apply CSS rules only when conditions (like screen width) are met.

```
/* Syntax */
@media (condition) { /* CSS rules here */ }

/* Width-based breakpoints (the most common) */
@media (max-width: 768px)  { /* tablet and below */ }
@media (min-width: 769px)  { /* desktop and above */ }
@media (max-width: 480px)  { /* phone only */ }

/* Range (modern syntax) */
@media (480px <= width <= 768px) { /* between phone and tablet */ }

/* Combining conditions */
@media (min-width: 600px) and (max-width: 900px) { }
@media (max-width: 600px) or (orientation: portrait) { }

/* Dark mode */
@media (prefers-color-scheme: dark) {
  body { background: #0d1117; color: #e6edf3; }
}

/* Print */
@media print {
  .no-print { display: none; }  /* hide navbars, ads */
  body { font-size: 12pt; }
}

/* High DPI screens */
@media (-webkit-min-device-pixel-ratio: 2), (min-resolution: 192dpi) {
  .logo { background-image: url('logo@2x.png'); }
}
```

Mobile-First Approach

Write base styles for mobile, then add complexity for larger screens with min-width queries. This is the industry standard.

```
/* ❌ Desktop-first (avoid) */
.sidebar {
  width: 300px;
  float: left;
```

```
/* ✅ Mobile-first (recommended) */
.sidebar {
  width: 100%;   /* mobile default */
}
```

```
}
@media (max-width: 768px) {
  .sidebar {
    width: 100%;
    float: none;
  }
}
```

```
@media (min-width: 768px) {
  .sidebar {
    width: 300px;  /* larger screens */
  }
}
```

Standard Breakpoints

```
/* Common breakpoint system (Tailwind CSS style) */
:root { --breakpoint-sm: 640px; --breakpoint-md: 768px; --breakpoint-lg: 1024px; --
breakpoint-xl: 1280px; }

/* Mobile base styles */
.container { padding: 0 16px; }
.grid-cols  { grid-template-columns: 1fr; }

/* Small: 640px+ (large phones, small tablets) */
@media (min-width: 640px) {
  .grid-cols { grid-template-columns: repeat(2, 1fr); }
}

/* Medium: 768px+ (tablets) */
@media (min-width: 768px) {
  .container  { padding: 0 32px; }
  .sidebar    { display: block; } /* show sidebar on tablet */
}

/* Large: 1024px+ (laptops) */
@media (min-width: 1024px) {
  .grid-cols  { grid-template-columns: repeat(3, 1fr); }
  .container  { max-width: 1024px; margin: 0 auto; }
}

/* XL: 1280px+ (desktop) */
@media (min-width: 1280px) {
  .grid-cols  { grid-template-columns: repeat(4, 1fr); }
  .container  { max-width: 1280px; }
}
```

Responsive Units

```
/* px — fixed pixels (use for borders, shadows, small fixed things) */
.border { border: 1px solid #ccc; }

/* % — relative to parent (use for widths) */
.half-width { width: 50%; }

/* rem — relative to ROOT font-size (use for font-sizes, spacing) */
/* If root is 16px: 1rem=16px, 1.5rem=24px, 2rem=32px */
h1 { font-size: 2.5rem; }  /* scales if user changes browser font size */

/* em — relative to CURRENT element's font-size (use for component-level) */
```

```
.btn { padding: 0.75em 1.5em; } /* scales with button's font-size */

/* vw/vh — viewport width/height */
.hero { height: 100vh; }           /* full viewport height */
.title { font-size: clamp(1.5rem, 4vw, 3rem); } /* responsive font size! */

/* clamp(min, ideal, max) — the magic responsive unit */
.container { width: clamp(300px, 90%, 1200px); }
h1 { font-size: clamp(1.8rem, 5vw, 4rem); }

/* min() and max() */
.sidebar { width: min(300px, 100%); } /* never wider than 300px or 100% */
```

📝 Project: Fully Responsive Navigation

```
/* Mobile: hamburger menu hidden; Desktop: horizontal nav */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 16px 24px;
  background: #1a1a2e;
  position: sticky;
  top: 0;
  z-index: 100;
}
.nav-logo { color: white; font-size: 1.5rem; font-weight: 800; }
.nav-links {
  display: none;   /* hidden on mobile */
  list-style: none;
  gap: 24px;
  margin: 0; padding: 0;
}
.nav-links a { color: rgba(255,255,255,0.8); text-decoration: none; }
.nav-links a:hover { color: white; }
.hamburger { display: flex; color: white; cursor: pointer; font-size: 1.5rem; }

@media (min-width: 768px) {
  .nav-links  { display: flex; }
  .hamburger  { display: none; }
}
```

CHAPTER 8

CSS Variables & the Cascade

Custom Properties, Specificity & Inheritance

CSS Custom Properties (Variables)

CSS variables let you store values and reuse them throughout your stylesheet. They make theming and maintenance dramatically easier.

```
/* Define variables on :root to make them globally available */
:root {
  /* Design tokens — the single source of truth */
  --color-primary:    #264DE4;
  --color-primary-dark: #1a3bc4;
  --color-accent:     #E44D26;
  --color-text:       #111827;
  --color-muted:      #6B7280;
  --color-bg:         #FFFFFF;
  --color-surface:    #F9FAFB;

  --font-sans: 'Inter', system-ui, sans-serif;
  --font-size-base: 1rem;

  --radius-sm:  4px;
  --radius-md:  8px;
  --radius-lg:  12px;
  --radius-full: 9999px;

  --shadow-sm: 0 1px 2px rgba(0,0,0,0.05);
  --shadow-md: 0 4px 6px rgba(0,0,0,0.07);
  --shadow-lg: 0 10px 15px rgba(0,0,0,0.1);

  --spacing-xs: 4px;  --spacing-sm: 8px;
  --spacing-md: 16px; --spacing-lg: 24px;
  --spacing-xl: 32px; --spacing-2xl: 48px;
}

/* Using variables */
.btn-primary {
  background: var(--color-primary);
  padding: var(--spacing-sm) var(--spacing-lg);
  border-radius: var(--radius-md);
  box-shadow: var(--shadow-sm);
}
.btn-primary:hover {
  background: var(--color-primary-dark);
}

/* var(--name, fallback) — use fallback if variable undefined */
.card {
  border-radius: var(--card-radius, var(--radius-md));
}
```

Dark Mode with CSS Variables

```
:root {
  --bg: #ffffff;
  --text: #111827;
  --surface: #f3f4f6;
}
@media (prefers-color-scheme: dark) {
  :root {
    --bg: #0d1117;
    --text: #e6edf3;
    --surface: #161b22;
  }
}
body {
  background: var(--bg);
  color: var(--text);
}
.card { background: var(--surface); }
/* ALL components automatically update — no CSS duplication! */
```

Variables in JavaScript

```
// Read a CSS variable
let primary = getComputedStyle(document.documentElement)
  .getPropertyValue('--color-primary').trim();

// Set a CSS variable dynamically
document.documentElement.style.setProperty('--color-primary', '#7C3AED');
// All elements using var(--color-primary) update instantly!

// Theme switcher
function setTheme(theme) {
  document.documentElement.setAttribute('data-theme', theme);
}
/* CSS: [data-theme='dark'] { --bg: #0d1117; --text: #e6edf3; } */
```

Specificity — Who Wins?

When multiple rules target the same element, specificity determines which styles apply. Higher specificity wins.

Specificity Score (a,b,c)

a = ID selectors (#header → 1,0,0 = 100 points)
b = Class, attribute, pseudo-class (.card, [type], :hover → 0,1,0 = 10 points)
c = Element, pseudo-element (p, ::before → 0,0,1 = 1 point)

HIGHER SCORE WINS regardless of source order.
!important overrides everything (use sparingly — it's a last resort).

```
p               { color: black; }   /* 0,0,1 = 1 pt  */
.intro          { color: blue;  }   /* 0,1,0 = 10 pt */
#main p         { color: green; }   /* 1,0,1 = 101 pt — WINS */
```

```
/* Specificity calculation examples */
a               /* 0,0,1 */
a:hover         /* 0,1,1 */
.nav a:hover    /* 0,2,1 */
#nav .link      /* 1,1,0 */

/* !important — overrides everything, even inline styles */
.override { color: red !important; }
/* Use only when you can't fix specificity otherwise */
```

The Cascade — How Rules Are Applied

```
/* CSS applies in this priority order (highest wins): */
/* 1. !important user agent styles (browser accessibility settings) */
/* 2. !important author styles (your CSS with !important) */
/* 3. Author inline styles (style='...') */
/* 4. Author styles (specificity, then source order) */
/* 5. User styles */
/* 6. Browser default styles */

/* Within same specificity — LATER rules win */
p { color: red; }
p { color: blue; }  /* blue wins — comes later */

/* Inheritance — some properties inherit from parent */
body {
  font-family: Arial;  /* inherits to all children */
  color: #111827;      /* inherits to all children */
}
/* Non-inherited: margin, padding, border, background, width, etc. */

/* Force inheritance */
.child { color: inherit; }

/* Reset to default */
.reset { all: unset; }
```

💡 TIP

Always organize CSS in this order: Custom Properties → Reset/Base → Typography → Layout → Components → Utilities. It prevents most specificity battles.

CHAPTER 9

Transitions & Animations

Bringing Your UI to Life

Animations make UIs feel responsive and polished. CSS transitions handle simple state changes (hover, focus). CSS animations handle complex multi-step sequences.

Transitions — Smooth State Changes

A transition animates a CSS property from one value to another over time.

```
/* Syntax: transition: property duration timing-function delay; */

/* Animate a button on hover */
.btn {
  background: #264DE4;
  color: white;
  padding: 12px 24px;
  border: none;
  border-radius: 6px;
  cursor: pointer;

  /* Transition: what to animate, how long, how */
  transition: background 0.2s ease, transform 0.2s ease, box-shadow 0.2s ease;
  /* OR: transition: all 0.2s ease; (animates ALL changing properties) */
}
.btn:hover {
  background: #1a3bc4;
  transform: translateY(-2px);
  box-shadow: 0 8px 16px rgba(38,77,228,0.3);
}
.btn:active {
  transform: translateY(0);
  box-shadow: none;
}

/* Timing functions */
/* ease        — slow start, fast middle, slow end (default) */
/* linear      — constant speed */
/* ease-in     — slow start */
/* ease-out    — slow end */
/* ease-in-out — slow start AND end */
/* cubic-bezier(x1,y1,x2,y2) — custom curve */
.spring { transition: transform 0.3s cubic-bezier(0.34, 1.56, 0.64, 1); }
/* Creates a spring/bounce effect */
```

Common Transition Patterns

```
/* Card hover lift */
.card {
  transition: transform 0.25s ease, box-shadow 0.25s ease;
```

```
}
.card:hover {
  transform: translateY(-4px);
  box-shadow: 0 12px 24px rgba(0,0,0,0.12);
}

/* Navigation link underline slide */
.nav-link {
  text-decoration: none;
  position: relative;
  padding-bottom: 2px;
}
.nav-link::after {
  content: '';
  position: absolute;
  bottom: 0; left: 0;
  width: 0;
  height: 2px;
  background: #264DE4;
  transition: width 0.3s ease;
}
.nav-link:hover::after { width: 100%; }

/* Smooth modal fade-in */
.modal {
  opacity: 0;
  transform: scale(0.95);
  transition: opacity 0.2s ease, transform 0.2s ease;
  pointer-events: none;
}
.modal.is-open {
  opacity: 1;
  transform: scale(1);
  pointer-events: auto;
}
```

CSS Animations with @keyframes

Keyframes define multi-step animations that can run automatically — no hover required.

```
/* Define the animation sequence */
@keyframes slideInUp {
  from {
    opacity: 0;
    transform: translateY(30px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* Apply the animation */
.hero-title {
  animation: slideInUp 0.6s ease-out forwards;
  /* name | duration | timing | fill-mode */
}
```

```
/* Stagger children (each delayed) */
.item:nth-child(1) { animation: slideInUp 0.5s 0.1s ease-out both; }
.item:nth-child(2) { animation: slideInUp 0.5s 0.2s ease-out both; }
.item:nth-child(3) { animation: slideInUp 0.5s 0.3s ease-out both; }
```

Keyframe Examples

```
/* Loading spinner */
@keyframes spin {
  to { transform: rotate(360deg); }
}
.spinner {
  width: 40px; height: 40px;
  border: 4px solid #e5e7eb;
  border-top-color: #264DE4;
  border-radius: 50%;
  animation: spin 0.8s linear infinite;
}

/* Pulse (like a heartbeat) */
@keyframes pulse {
  0%, 100% { transform: scale(1);   opacity: 1; }
  50%       { transform: scale(1.1); opacity: 0.8; }
}
.live-dot { animation: pulse 2s ease-in-out infinite; }

/* Fade in */
@keyframes fadeIn {
  from { opacity: 0; }
  to   { opacity: 1; }
}

/* Shake (error feedback) */
@keyframes shake {
  0%, 100% { transform: translateX(0); }
  20%, 60% { transform: translateX(-8px); }
  40%, 80% { transform: translateX(8px); }
}
.input-error { animation: shake 0.4s ease; }

/* Skeleton loading placeholder */
@keyframes shimmer {
  from { background-position: -200% 0; }
  to   { background-position:  200% 0; }
}
.skeleton {
  background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%);
  background-size: 200% 100%;
  animation: shimmer 1.5s infinite;
  border-radius: 4px;
}
```

Animation Properties Reference

Property	Values	Description

`animation-name`	keyframe-name	Which @keyframes to run
`animation-duration`	0.5s, 1000ms	How long one cycle takes
`animation-timing-function`	ease, linear, ease-in-out	Speed curve
`animation-delay`	0.2s, -1s	Wait before starting (negative = start mid-animation)
`animation-iteration-count`	1, 2, infinite	How many times to repeat
`animation-direction`	normal, reverse, alternate	normal→, reverse←, alternate↔
`animation-fill-mode`	none, forwards, backwards, both	What to keep after animation ends
`animation-play-state`	running, paused	Pause/resume via JS

💡 TIP

Use transform and opacity for animations — they run on the GPU (composited layer) and don't trigger layout. Avoid animating width, height, top, left, margin — they're slow.

CHAPTER 10

Transforms & Filters

2D/3D Transforms, Shadows & Visual Effects

2D Transforms

Transforms move, scale, rotate, and skew elements without affecting the document flow. They're GPU-accelerated and won't cause layout reflow.

```
/* translate — move element */
.move-right  { transform: translateX(50px); }
.move-down   { transform: translateY(20px); }
.move-both   { transform: translate(50px, 20px); }
.center-abs  { transform: translate(-50%, -50%); } /* center trick */

/* scale — resize element */
.bigger  { transform: scale(1.5);     }  /* 150% */
.smaller { transform: scale(0.8);     }  /* 80% */
.wide    { transform: scaleX(2);      }  /* double width */
.flip    { transform: scaleX(-1);     }  /* horizontal flip */

/* rotate — spin element */
.slight  { transform: rotate(5deg);   }
.quarter { transform: rotate(90deg);  }
.flip-v  { transform: rotate(180deg); }

/* skew — slant element */
.slanted { transform: skewX(-15deg);  }

/* Combining transforms */
.combined { transform: rotate(45deg) scale(1.2) translateX(10px); }

/* transform-origin — change pivot point */
.tooltip { transform-origin: bottom center; } /* scale from bottom */
.badge   { transform-origin: top right; }
```

3D Transforms

```
/* Enable 3D on the parent */
.card-wrapper {
  perspective: 1000px;        /* depth of 3D space */
}

/* 3D flip card effect */
.card {
  position: relative;
  width: 300px; height: 200px;
  transform-style: preserve-3d;
  transition: transform 0.6s ease;
}
.card:hover {
  transform: rotateY(180deg); /* flip horizontally */
```

```
}
.card-front,
.card-back {
  position: absolute; inset: 0;
  backface-visibility: hidden; /* hide back when facing away */
  border-radius: 12px;
  display: flex; align-items: center; justify-content: center;
}
.card-front { background: #264DE4; color: white; }
.card-back  { background: #E44D26; color: white; transform: rotateY(180deg); }
```

box-shadow

```
/* Syntax: offset-x offset-y blur spread color */
.shadow-sm   { box-shadow: 0 1px 2px rgba(0,0,0,0.05); }
.shadow-md   { box-shadow: 0 4px 6px rgba(0,0,0,0.07); }
.shadow-lg   { box-shadow: 0 10px 15px rgba(0,0,0,0.10); }
.shadow-xl   { box-shadow: 0 20px 25px rgba(0,0,0,0.15); }

/* Colored glow shadow */
.glow-blue  { box-shadow: 0 0 20px rgba(38, 77, 228, 0.5); }
.glow-red   { box-shadow: 0 0 20px rgba(228, 77, 38, 0.5); }

/* Inner shadow */
.inset      { box-shadow: inset 0 2px 4px rgba(0,0,0,0.1); }

/* Multiple shadows */
.layered    { box-shadow: 0 2px 4px rgba(0,0,0,0.06), 0 8px 16px rgba(0,0,0,0.08); }

/* Bottom-only shadow (no top/sides) */
.bottom-only { box-shadow: 0 8px 8px -8px rgba(0,0,0,0.3); }
```

CSS Filters

Filters apply visual effects to images and elements — like Photoshop, but in CSS.

```
/* Apply to any element (img, div, etc.) */

/* Blur */
.blur-sm { filter: blur(4px); }
.blur-bg { filter: blur(12px); }  /* frosted glass background */

/* Brightness and contrast */
.dim     { filter: brightness(0.7); }
.bright  { filter: brightness(1.3); }
.high-c  { filter: contrast(1.5); }

/* Grayscale */
.bw          { filter: grayscale(100%); }
.bw:hover    { filter: grayscale(0); transition: filter 0.3s; }

/* Hue rotate — change color without changing image */
.hue-shift   { filter: hue-rotate(90deg); }
```

```
/* Sepia / vintage */
.vintage     { filter: sepia(50%) contrast(1.1) brightness(1.05); }

/* Multiple filters */
.instagram   { filter: contrast(1.2) brightness(1.1) saturate(1.3); }

/* Invert */
.inverted    { filter: invert(100%); }

/* Drop shadow on transparent images (unlike box-shadow) */
.png-shadow  { filter: drop-shadow(4px 4px 8px rgba(0,0,0,0.4)); }
```

Clip-path — Custom Shapes

```
/* Clip element to a shape */

/* Circle */
.avatar { clip-path: circle(50%); }

/* Polygon */
.arrow   { clip-path: polygon(0 0, 100% 50%, 0 100%); }
.diagonal { clip-path: polygon(0 0, 100% 0, 100% 80%, 0 100%); }

/* Inset (rectangle with optional rounded corners) */
.inset   { clip-path: inset(10px 20px round 8px); }

/* Animate clip-path on hover */
.btn-clip {
  clip-path: inset(0 100% 0 0);   /* hidden */
  transition: clip-path 0.3s ease;
}
.btn-clip:hover { clip-path: inset(0 0% 0 0); } /* revealed */

/* Online tool: https://bennettfeely.com/clippy */
```

CHAPTER 11

Pseudo-Elements & Advanced Selectors

::before, ::after & Beyond

Pseudo-elements create virtual elements via CSS — no extra HTML needed. They're used for decorations, icons, clearfixes, and visual tricks.

::before and ::after

```
/* ::before and ::after insert content before/after an element's content */
/* They must have a 'content' property (can be empty string '') */

/* Quotation marks */
.quote::before { content: "\201C"; } /* open quote */
.quote::after  { content: "\201D"; } /* close quote */

/* Label badge */
.new-badge {
  position: relative;
}
.new-badge::after {
  content: 'NEW';
  position: absolute;
  top: -8px; right: -8px;
  background: #E44D26;
  color: white;
  font-size: 10px;
  font-weight: 700;
  padding: 2px 6px;
  border-radius: 9999px;
}

/* Clearfix (fix float collapse) */
.clearfix::after {
  content: '';
  display: table;
  clear: both;
}

/* Decorative line using ::before */
.section-title {
  position: relative;
  padding-bottom: 12px;
}
.section-title::after {
  content: '';
  position: absolute;
  bottom: 0; left: 0;
  width: 60px; height: 4px;
  background: #264DE4;
  border-radius: 2px;
}
```

Practical Pseudo-Element Patterns

```
/* Custom checkbox using ::before */
.checkbox-input { display: none; }
.checkbox-label {
  display: flex;
  align-items: center;
  gap: 10px;
  cursor: pointer;
}
.checkbox-label::before {
  content: '';
  width: 20px; height: 20px;
  border: 2px solid #ccc;
  border-radius: 4px;
  transition: background 0.2s, border-color 0.2s;
}
.checkbox-input:checked + .checkbox-label::before {
  background: #264DE4;
  border-color: #264DE4;
  content: '\2713'; /* ✓ */
  color: white;
  font-size: 14px;
  display: flex; align-items: center; justify-content: center;
}

/* Tooltip using ::before ::after */
.has-tooltip {
  position: relative;
  cursor: pointer;
}
.has-tooltip::before {  /* tooltip text */
  content: attr(data-tooltip); /* reads from HTML attribute! */
  position: absolute;
  bottom: 100%; left: 50%;
  transform: translateX(-50%);
  background: #111;
  color: white;
  padding: 6px 10px;
  border-radius: 4px;
  font-size: 13px;
  white-space: nowrap;
  opacity: 0;
  transition: opacity 0.2s;
  pointer-events: none;
}
.has-tooltip:hover::before { opacity: 1; }
```

::first-line, ::first-letter, ::selection, ::placeholder

```
/* Style first letter (drop cap) */
p::first-letter {
  font-size: 3rem;
  font-weight: 900;
  float: left;
  line-height: 1;
  margin-right: 6px;
  color: #264DE4;
```

```
}

/* Style first line only */
p::first-line {
  font-variant: small-caps;
  letter-spacing: 0.05em;
}

/* Style text selection */
::selection {
  background: #264DE4;
  color: white;
}

/* Style input placeholder */
input::placeholder {
  color: #9CA3AF;
  font-style: italic;
}
input:focus::placeholder { opacity: 0; } /* hide on focus */
```

Advanced Selector Combinations

```
/* :has() — parent selector (modern CSS!) */
.card:has(img) { padding: 0; }           /* card WITH an image */
form:has(:invalid) { border: red 2px solid; } /* form with invalid input */

/* :is() — match any of these */
:is(h1, h2, h3) { font-weight: 800; }
:is(.card, .panel, .modal) > p { margin: 0; }

/* :where() — like :is() but zero specificity */
:where(h1, h2, h3, h4) { line-height: 1.2; }

/* Focus visible — keyboard focus only */
.btn:focus-visible {
  outline: 3px solid #264DE4;
  outline-offset: 2px;
}
.btn:focus:not(:focus-visible) {
  outline: none;   /* hide ugly outline on mouse click */
}

/* :empty — elements with no content */
.error:empty { display: none; }

/* :target — element matching URL hash */
:target { background: #fef3c7; }
/* If URL is page.html#section2, #section2 gets the style */
```

CHAPTER 12

CSS Architecture & Best Practices

BEM, Performance & The Modern Toolkit

BEM — Block, Element, Modifier

BEM is a naming convention that makes CSS scalable and readable. It's the most widely-used CSS naming methodology.

```
/*
 * Block    — standalone component  (.card)
 * Element  — part of a block       (.card__title)
 * Modifier — variant of block/el   (.card--featured, .card__title--large)
 */

/* HTML:
  <article class='card card--featured'>
    <img class='card__image' src='...' alt='...'>
    <div class='card__body'>
      <h2 class='card__title card__title--large'>Title</h2>
      <p class='card__description'>Text here</p>
      <a class='card__link btn btn--primary' href='#'>Read More</a>
    </div>
  </article>
*/

/* CSS: never nest selectors deeply */
.card { border-radius: 12px; overflow: hidden; }
.card--featured { border: 2px solid #264DE4; }
.card__image  { width: 100%; aspect-ratio: 16/9; object-fit: cover; }
.card__body   { padding: 20px; }
.card__title  { font-size: 1.2rem; margin-bottom: 8px; }
.card__title--large { font-size: 1.6rem; }
.card__description { color: #6B7280; }
.card__link   { display: inline-block; margin-top: 16px; }
```

The display Property

```
/* block — full width, starts on new line */
div, p, h1, section { display: block; }

/* inline — flows with text, no width/height */
span, a, strong, em { display: inline; }

/* inline-block — inline flow, but accepts width/height/padding */
.badge { display: inline-block; padding: 4px 8px; }

/* flex — flexbox container */
.nav { display: flex; gap: 16px; }

/* grid — grid container */
.layout { display: grid; }
```

```
/* none — completely remove from layout (invisible + no space) */
.hidden { display: none; }

/* contents — element becomes transparent (children take its place) */
.wrapper { display: contents; }

/* table display values */
.data      { display: table; width: 100%; }
.data-row  { display: table-row; }
.data-cell { display: table-cell; padding: 8px; }
```

Overflow

```
/* overflow controls content that exceeds element bounds */
.el { overflow: visible; }  /* default — spills outside */
.el { overflow: hidden;  }  /* clip — hide overflowing content */
.el { overflow: scroll;  }  /* always show scrollbar */
.el { overflow: auto;    }  /* scrollbar only when needed */

/* Separate axes */
.horizontal-scroll { overflow-x: auto; overflow-y: hidden; }

/* Clamp text to N lines */
.line-clamp {
  display: -webkit-box;
  -webkit-line-clamp: 3;        /* max 3 lines */
  -webkit-box-orient: vertical;
  overflow: hidden;
}

/* Text overflow on single line */
.truncate {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
  max-width: 200px;
}
```

CSS Performance

Performance Rules

1. Use transform/opacity for animations — GPU-accelerated, no layout reflow
2. Use will-change: transform on elements you'll animate (hints to browser)
3. Minimize selector depth — .card .title .text is slower than .card-text
4. Use contain: layout | paint | strict on isolated components
5. Avoid !important — it makes override calculations expensive
6. Load critical CSS inline, defer rest
7. Use @media queries to conditionally load CSS

```
/* will-change — warn browser in advance */
```

```
.animated-card {
  will-change: transform;      /* browser creates composite layer */
}
/* Remove after animation finishes to free memory */

/* contain — isolate component for render performance */
.widget {
  contain: layout;    /* changes inside don't affect outside */
}

/* @layer — cascade layers for better organization */
@layer reset, base, components, utilities;

@layer reset {
  * { margin: 0; padding: 0; box-sizing: border-box; }
}
@layer base {
  body { font-family: system-ui; }
}
@layer components {
  .btn { padding: 10px 20px; border-radius: 6px; }
}
@layer utilities {
  .mt-4 { margin-top: 16px; }
  .text-center { text-align: center; }
}
```

CSS Reset / Normalize

```
/* Modern CSS reset (Josh Comeau's approach) */
*, *::before, *::after {
  box-sizing: border-box;
}
* {
  margin: 0;
}
html {
  scroll-behavior: smooth;
}
body {
  line-height: 1.5;
  -webkit-font-smoothing: antialiased;
}
img, picture, video, canvas, svg {
  display: block;
  max-width: 100%;
}
input, button, textarea, select {
  font: inherit;  /* inputs don't inherit font by default! */
}
p, h1, h2, h3, h4, h5, h6 {
  overflow-wrap: break-word;
}
```

📝 Final Project: Complete Landing Page

```
/* Full landing page styles — combine everything learned */

/* 1. Variables */
:root {
  --clr-primary: #264DE4;  --clr-accent: #E44D26;
  --clr-text: #111827;     --clr-muted: #6B7280;
  --clr-bg: #fff;          --clr-surface: #F9FAFB;
  --radius: 12px;          --shadow: 0 4px 6px rgba(0,0,0,0.07);
  --max-w: 1200px;         --font: 'Inter', system-ui, sans-serif;
}

/* 2. Reset */
*, *::before, *::after { box-sizing: border-box; margin: 0; }
body { font-family: var(--font); color: var(--clr-text); line-height: 1.6; }
img  { display: block; max-width: 100%; }

/* 3. Layout utilities */
.container { max-width: var(--max-w); margin: 0 auto; padding: 0 24px; }
.section   { padding: 80px 0; }
.section:nth-child(even) { background: var(--clr-surface); }

/* 4. Navigation */
.nav {
  position: sticky; top: 0; z-index: 100;
  display: flex; justify-content: space-between; align-items: center;
  padding: 16px 32px;
  background: rgba(255,255,255,0.95);
  backdrop-filter: blur(12px);
  box-shadow: 0 1px 0 rgba(0,0,0,0.08);
}
.nav-links { display: flex; gap: 32px; list-style: none; }
@media (max-width: 768px) { .nav-links { display: none; } }

/* 5. Hero */
.hero {
  min-height: 100vh;
  display: flex; align-items: center; justify-content: center;
  text-align: center;
  background: linear-gradient(135deg, #0f0c29, #302b63, #24243e);
  padding: 80px 24px;
}
.hero h1 { font-size: clamp(2rem, 6vw, 5rem); color: white; font-weight: 900; }
.hero p  { font-size: clamp(1rem, 2vw, 1.3rem); color: rgba(255,255,255,0.75);
margin-top: 20px; }

/* 6. Cards grid */
.cards { display: grid; grid-template-columns: repeat(auto-fill, minmax(280px,
1fr)); gap: 24px; }
.card {
  background: var(--clr-bg);
  border: 1px solid #e5e7eb;
  border-radius: var(--radius);
  padding: 28px;
  box-shadow: var(--shadow);
  transition: transform 0.25s ease, box-shadow 0.25s ease;
}
.card:hover { transform: translateY(-4px); box-shadow: 0 12px 24px rgba(0,0,0,0.12);
}
```

```
/* 7. Button system */
.btn {
  display: inline-flex; align-items: center; gap: 8px;
  padding: 12px 24px; border-radius: var(--radius);
  font-weight: 600; font-size: 0.95rem;
  text-decoration: none; border: none; cursor: pointer;
  transition: all 0.2s ease;
}
.btn-primary  { background: var(--clr-primary); color: white; }
.btn-primary:hover  { background: #1a3bc4; transform: translateY(-1px); }
.btn-secondary { background: transparent; color: var(--clr-primary); border: 2px
solid var(--clr-primary); }
.btn-secondary:hover { background: var(--clr-primary); color: white; }
```

CHAPTER 13

Quick Reference

CSS Cheatsheet

Properties Quick Reference

Category	Property	Key Values
Text	font-size	px, rem, em, clamp(min,val,max)
Text	font-weight	100–900, bold, bolder
Text	font-family	name, serif, sans-serif, monospace
Text	line-height	1.5 (unitless — relative to font-size)
Text	text-align	left, center, right, justify
Text	color	hex, rgb(), hsl(), named
Box	width / height	px, %, rem, vw, vh, min/max, auto
Box	margin	shorthand: T R B L | auto | %
Box	padding	shorthand: T R B L | %
Box	border	width style color | none
Box	border-radius	px, % (50% = circle)
Box	box-shadow	x y blur spread color, inset
Layout	display	block, inline, flex, grid, none
Layout	position	static, relative, absolute, fixed, sticky
Layout	z-index	number (higher = on top)
Layout	overflow	visible, hidden, auto, scroll
Flex	justify-content	flex-start, center, space-between, space-around
Flex	align-items	flex-start, center, flex-end, stretch
Flex	flex	1, 0 0 200px, grow shrink basis
Flex	gap	px, rem
Grid	grid-template-columns	1fr, px, repeat(3,1fr), auto-fill minmax
Grid	grid-column	1/-1, span 2
Grid	grid-area	named-area
Anim	transition	property duration timing delay
Anim	animation	name duration timing delay count direction fill
Anim	transform	translate, scale, rotate, skew
Visual	opacity	0–1
Visual	filter	blur, brightness, contrast, grayscale
Visual	background	color, image, gradient, shorthand

Flexbox Cheatsheet

```
/* CONTAINER */                              /* ITEM */
.flex {                                      .item {
  display: flex;                               flex: 1;      /* grow equally */
  flex-direction: row;        /* → */          flex: 0 0 200px; /* fixed */
  flex-wrap: wrap;            /* multiline */  align-self: center;
  justify-content: center;   /* main axis */   order: -1;    /* reorder */
  align-items: center;       /* cross axis */  margin-left: auto; /* push right */
  gap: 16px;                                 }
}
```

Grid Cheatsheet

```
/* CONTAINER */                              /* ITEM */
.grid {                                      .item {
  display: grid;                               grid-column: 1 / 3;  /* span */
  grid-template-columns: repeat(3, 1fr);       grid-column: span 2;
  grid-template-rows: auto;                    grid-row: 1 / 3;
  gap: 20px;                                   align-self: center;
  justify-items: stretch;                      justify-self: end;
  align-items: stretch;                      }
}
/* Responsive: repeat(auto-fill, minmax(250px, 1fr)) */
```

Common Patterns Cheatsheet

```
/* Center anything */
.center { display:flex; justify-content:center; align-items:center; }

/* Responsive card grid */
.grid  { display:grid; grid-template-columns:repeat(auto-fill,minmax(280px,1fr));
gap:24px; }

/* Truncate text */
.trunc { white-space:nowrap; overflow:hidden; text-overflow:ellipsis; max-
width:200px; }

/* Aspect ratio box */
.ratio { aspect-ratio: 16/9; }

/* Visually hidden (accessible) */
.sr-only { position:absolute; width:1px; height:1px; overflow:hidden;
clip:rect(0,0,0,0); }

/* Full-bleed image */
.cover { width:100%; height:100%; object-fit:cover; }

/* Smooth scroll */
html   { scroll-behavior:smooth; }

/* Prevent layout shift from scrollbar */
body   { overflow-y:scroll; }
```

```
/* Focus ring (accessibility) */
:focus-visible { outline:3px solid #264DE4; outline-offset:2px; }

/* Loading skeleton */
@keyframes shimmer { from{background-position:-200% 0} to{background-position:200%
0} }
.skeleton { background:linear-gradient(90deg,#f0f0f0 25%,#e0e0e0 50%,#f0f0f0 75%);
background-size:200%;animation:shimmer 1.5s infinite; }
```

🎓 You've Completed CSS from Beginner to Professional!

You now know:

- ✓ How to write and link CSS — external, internal, inline
- ✓ All selector types — element, class, ID, attribute, pseudo-class
- ✓ The Box Model — margin, border, padding, box-sizing
- ✓ Colors — hex, rgb, hsl, rgba, CSS variables
- ✓ Typography — fonts, sizes, line-height, text effects
- ✓ Backgrounds — color, image, gradient, attachment
- ✓ Flexbox — one-dimensional layouts
- ✓ CSS Grid — two-dimensional page layouts
- ✓ Positioning — relative, absolute, fixed, sticky
- ✓ Responsive design — media queries, rem/vw, clamp()
- ✓ CSS Variables — design tokens, dark mode
- ✓ Cascade & Specificity — who wins when rules conflict
- ✓ Transitions & Animations — @keyframes, timing functions
- ✓ Transforms — translate, scale, rotate, 3D effects
- ✓ Pseudo-elements — ::before, ::after, practical patterns
- ✓ BEM, Performance, Reset — professional CSS architecture

Next: Learn a CSS framework (Tailwind CSS), a preprocessor (Sass), and combine CSS with JavaScript for dynamic UIs!